AuraMart

Project Execution Plan

Step by step from day one to v1.0, with Scrum sprint planning

Solo-developer edition · 19 Oct 2026 → 18 Apr 2027 · plan written 15 Oct 2026

At a glance

Start	Monday 19 Oct 2026 — Sprint 0
Finish	Sunday 18 Apr 2027 — v1.0.0 released
Duration	26 weeks: Sprint 0 (1 week) + Sprints 1–12 (2 weeks each) + Sprint 13 release (1 week)
Backlog	9 epics · 66 user stories · 240 story points
Capacity	15 hours a week: 2 h Monday–Friday, 5 h Saturday, Sunday off ≈ 18 points per 2-week sprint
Milestones	7 milestone releases, from v0.1.0 (Catalog API, 8 Nov) to v1.0.0 (18 Apr)
Tools	GitHub repo, GitHub Projects board and GitHub Actions CI — all free

How to use this document

Read Sections 1–5 once: they explain where to start, how Scrum works when you are a team of one, the rules every story must follow and the release plan. Section 6 is your first week, day by day. Sections 7–9 repeat every sprint: how to run sprint planning, the plan for each sprint, and how to track progress. The rest is reference. Use it together with the **Project Overview** (architecture, tech stack, pricing and features); both are in the AuraMart folder's 1-docs.

> **Your first action — before Monday 19 October**
>
> Block 15 hours a week in your calendar (2 h on weekdays, 5 h on Saturday, Sunday off), put the **AuraMart** folder in your Downloads folder, and read Sections 1–4. Sprint 0 starts on **Monday 19 October 2026**; the **Phase 1 Build Guide** takes you through it and Sprint 1 step by step.

Contents

1. Where to start . . . 3

2. Product vision & scope . . . 4

3. Scrum for a team of one . . . 5

4. Rules of the game . . . 7

5. Release plan . . . 8

6. Sprint 0: your first week . . . 9

Sprint 0 · Setup & ways of working . . . 9

7. Sprint planning (with Sprint 1 filled in) . . . 11

8. Sprint-by-sprint plan . . . 13

Sprint 1 · Catalog service . . . 14

Sprint 2 · API Gateway & real-infrastructure tests . . . 15

Sprint 3 · Speed & search . . . 16

Sprint 4 · Orders & reliable events . . . 17

Sprint 5 · Checkout saga . . . 18

Sprint 6 · Notifications, idempotency & DLT . . . 19

Sprint 7 · Semantic search . . . 20

Sprint 8 · AI shopping assistant . . . 21

Sprint 9 · Storefront I: browse, search, cart . . . 22

Sprint 10 · Storefront II: checkout & AI chat . . . 23

Sprint 11 · Run everything & observe it . . . 24

Sprint 12 · Secure, ship & scale . . . 25

Sprint 13 · Release & portfolio launch . . . 26

9. Tracking progress . . . 27

10. Risks and how to handle them . . . 29

11. Documentation plan . . . 30

12. Launch and beyond . . . 31

Appendix A. Git workflow . . . 32

1. Where to start

A real project does not start with code. It starts by agreeing what to build and why, then turning that into a prioritised list of small pieces of work, and only then building it in short, repeating cycles. For AuraMart the first three steps are already done — this plan is their result.

#	Step	What you do	Output	Where · status
1	Vision & scope	Decide what AuraMart is, who it is for and what is not in v1.0	Vision, personas, scope	Section 2 · done
2	Product backlog	Split the product into epics and user stories with acceptance criteria and estimates	9 epics, 66 stories, 240 points	Sections 6–8 · done
3	Release plan	Put the stories into dated sprints with milestones	14 sprints, 7 milestones	Section 5 · done
4	Sprint 0	Install tools; set up the repo, board, CI and infrastructure	A project ready to build	Section 6 · 19 Oct – 25 Oct 2026
5	Sprints 1–12	Repeat the sprint cycle every two weeks	A working, tested release every sprint	Sections 7–9
6	Release sprint	Bug bash, documentation, demo video	v1.0.0	Sprint 13 · 12 Apr – 18 Apr 2027
7	After launch	Present, maintain and extend the project	Portfolio, interviews	Section 12

The sprint cycle

Every sprint repeats the same five events. The timebox never changes: if work isn't finished, the sprint still ends on time and the unfinished story goes back to the backlog to be re-planned.



> **The whole method in one line**
>
> Plan a little, build a little, show it, improve how you work — then repeat. Every two weeks you have something that runs, a release tag, and one concrete improvement to how you work.

2. Product vision & scope

Vision

> *For online shoppers who want help choosing what to buy, AuraMart is an online store with an AI shopping assistant that recommends real products from the catalogue and tracks orders. Unlike a basic CRUD shop, it is built like a production system: event-driven microservices that stay correct when payments fail, messages repeat or services go down.*

Personas — who you are building for

Persona	Who	Needs	Stories that serve them
Priya — shopper	27, software engineer; shops on her phone in short breaks	Quick, trustworthy recommendations; a checkout she doesn't have to think about	Search, AI assistant, cart, checkout, order tracking
Ravi — store admin	Runs the catalogue and stock levels	Edits that appear instantly; stock that is never oversold	Product management, cache eviction, stock reservation
You — developer / operator	Builds and runs AuraMart	See what broke and why; recover without losing data	Request IDs, DLT, tracing, dashboards, runbook

Scope — what is in and out of v1.0

In v1.0	Not in v1.0 (goes to 'Later' or never)
Catalogue, keyword and semantic search	Real payments (a simulated payment is used)
Cart, checkout and order tracking	Multiple sellers / marketplace
AI assistant: RAG, tools, memory, streaming	Shipping and delivery tracking
Notifications (log or local email)	Native mobile apps (the web app is responsive)
Admin through the API and Swagger UI	Admin screens in the UI
Login with Keycloak (shopper and admin roles)	Multiple languages or currencies
Tracing, metrics, resilience, CI, Kubernetes (local)	Public cloud hosting (an optional AWS demo only)

What 'finished' means for v1.0

- All four end-to-end flows (A–D in the Project Overview PDF) work through the UI.
- One checkout can be followed in a single trace across every service and Kafka.
- CI is green on main and backend line coverage is at least 70%.
- The whole system starts with one command on a 16 GB laptop and also deploys to kind.
- Documentation, ADRs and a 5-minute demo video are published.

3. Scrum for a team of one

Scrum normally has three roles. On AuraMart you play all three. The trick is knowing which hat you are wearing, because each one asks a different question.

Hat	Asks	Responsible for	When you wear it
Product Owner	“What is most valuable next?”	Owns and orders the backlog; writes acceptance criteria; accepts work as Done	Sprint planning, refinement, sprint review
Scrum Master	“Is the process helping or hurting?”	Protects timeboxes and focus; removes blockers; runs the retro; stops scope creep	Retrospective; whenever you are stuck for over an hour
Developer	“How do we build this well?”	Builds, tests and documents each story to the Definition of Done	Every working session

The five events

Event	When	Timebox	Output
Sprint planning	Monday, day 1	60 min	Sprint goal, committed stories, task list
Daily standup	Start of each session	5 min	DEVLOG.md entry
Backlog refinement	Saturday, week 1	30 min	Next sprint's stories meet the Definition of Ready
Sprint review	Saturday, week 2	30 min	Demo video, release tag, README update
Retrospective	Straight after the review	20 min	One improvement action for the next sprint

The three artifacts

- **Product backlog** — every story, in priority order (the Backlog view of your board).
- **Sprint backlog** — the sprint goal, the committed stories and their tasks (the Current sprint view).
- **Increment** — working, tested software, tagged as a release at every sprint review.

Your weekly rhythm (15 hours)

Day	Hours	What
Monday	2 h	Build. On the first Monday of a sprint, the first hour is sprint planning.
Tuesday – Friday	2 h a day	Build. Start every session with a 5-minute standup note.
Saturday	5 h	Deep work on the hardest story. Week 1: + 30 min refinement. Week 2: review + retro.
Sunday	0 h	Rest. Over 26 weeks, this is what stops burnout.

Capacity per 2-week sprint: 30 hours − about 3 hours of Scrum events = 27 hours of building ≈ 18 story points (using the starting guide of about 1.5 hours per point — see Section 4).

Setting up the board (GitHub Projects)

1. Create a project with the Board layout and link it to the auramart repository.
2. Status column values: Backlog, Ready, In progress, In review, Done.
3. Add fields: Sprint (Iteration, 2 weeks — make Sprint 0 a 1-week iteration from 19 Oct), Points (number), Epic (single select E0–E8), Priority (Must / Should / Could).

4. Create three views: Current sprint (board filtered to the current iteration), Backlog (table grouped by Epic), Roadmap (by Sprint).
5. One GitHub issue per story, titled with its ID — e.g. “AM-7 Create, update and deactivate products” — with the acceptance criteria as a checklist.
6. WIP limit: at most 2 cards In progress. Finish before you start.

Your Scrum Master rules

- The sprint length is fixed. Never extend a sprint — unfinished stories go back to the backlog and are re-estimated.
- A story is either Done or not. Ninety per cent done counts as zero in the velocity.
- Stuck for more than an hour? Write the problem down, read the official docs, ask (a forum, a friend, Claude), or switch task and come back.
- New ideas go to a 'Later' column, never into the running sprint. Only sprint planning adds work.
- Every sprint ends with something that runs and can be demonstrated.

4. Rules of the game

Definition of Ready — a story may enter a sprint only when…

- ☐ It is written as “As a …, I want …, so that …”.
- ☐ Its acceptance criteria are specific and testable.
- ☐ It is estimated at 8 points or less (a 13 must be split first).
- ☐ Its dependencies are Done, or in the same sprint and earlier in the order.
- ☐ You know how you will demo it.
- ☐ Anything you need to learn is noted, with a link to the docs or tutorial.

Definition of Done — a story is Done only when…

- ☐ The code is merged to main through a pull request and CI is green.
- ☐ You have self-reviewed it with the PR checklist, after a break, with fresh eyes.
- ☐ Unit tests are written, plus an integration test where a database, cache or Kafka is involved.
- ☐ Every acceptance criterion has been demonstrated (.http file, curl or UI).
- ☐ API changes are visible in Swagger and any new configuration is documented.
- ☐ Docs are updated: README, an ADR if a decision was made, events.md if an event changed.
- ☐ No secrets are committed and there is no TODO without an issue.
- ☐ The card is in Done and the DEVLOG is updated.

Estimating with story points

Points	Meaning	AuraMart example
1	Tiny and clear — an hour or two	Swagger title and health check (AM-11)
2	Small; you know exactly how	Price and stock status (AM-9)
3	Medium; a few unknowns	Browse with filter and paging (AM-8)
5	Large; needs some learning	Typo-tolerant search (AM-20)
8	Very large; a new concept	Transactional outbox (AM-26)
13	Too big — split it before it enters a sprint	“Build the AI assistant”

Story points measure size and uncertainty, not hours. As a starting guide only, assume about 1.5 hours per point for a learner. After Sprint 2, drop the guess and plan with your measured **velocity**: the average number of points you actually completed over the last three sprints.

Priorities

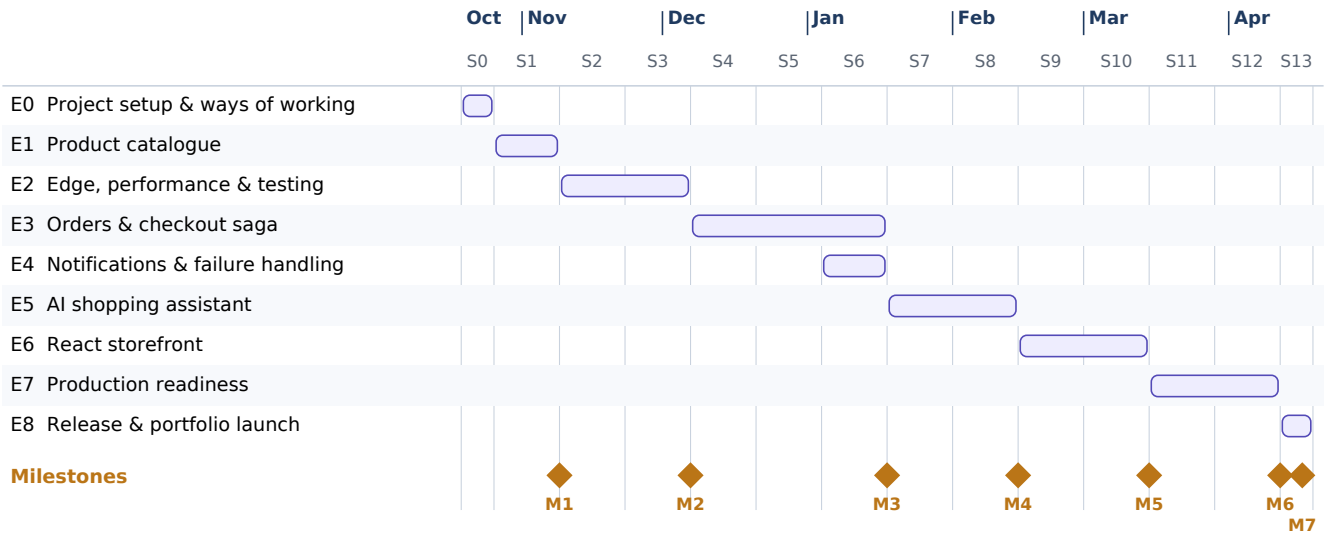
Must = the sprint goal fails without it. **Should** = important, but the goal survives without it. **Could** = stretch work, the first thing to drop. If a sprint runs late, drop Could, then Should — never quality.

Git workflow in one paragraph

One branch per story (`feature/AM-7-product-crud`), Conventional Commit messages that mention the story ID, a pull request for every change, squash merge, and a release tag at every sprint review. Full details and templates are in Appendix A.

5. Release plan

9 epics delivered across 14 sprints. Every sprint ends with a tagged release; 7 of them are milestones worth sharing (a GitHub release and a LinkedIn post).



Oct 2026 – Apr 2027 · bars = the sprints that deliver each epic · ◆ = milestone at the end of a sprint.

Milestones

Milestone	Date	What you can show
M1 · v0.1.0	8 Nov 2026	Catalog API with tests, Swagger and CI
M2 · v0.3.0	6 Dec 2026	One gateway, Redis caching, typo-tolerant search
M3 · v0.6.0	17 Jan 2027	Full checkout saga with compensation, retries and DLT
M4 · v0.8.0	14 Feb 2027	AI assistant with RAG, tool calling and memory
M5 · v0.10.0	14 Mar 2027	A usable store in the browser — the MVP
M6 · v0.12.0	11 Apr 2027	Tracing, dashboards, login and Kubernetes
M7 · v1.0.0	18 Apr 2027	v1.0 with documentation and a demo video

Epics

Epic	Name	Sprints	Stories	Points
E0	Project setup & ways of working	S0	5	10
E1	Product catalogue	S1	7	17
E2	Edge, performance & testing	S2–S3	10	35
E3	Orders & checkout saga	S4–S6	9	41
E4	Notifications & failure handling	S6	4	16
E5	AI shopping assistant	S7–S8	10	37
E6	React storefront	S9–S10	10	38
E7	Production readiness	S11–S12	7	38
E8	Release & portfolio launch	S13	4	8
	Total	S0–S13	66	240

6. Sprint 0: your first week

Sprint 0 is a one-week setup sprint. It produces no features — it makes everything after it faster. Here is the week, day by day; the Phase 1 Build Guide has every command for it.

Day	Focus	Story
Mon 19 Oct	Install JDK 21, Maven, Git, Docker Desktop, an IDE and Node.js LTS; check every version.	AM-1
Tue 20 Oct	Copy the starter code and the setup kit to C:\dev\auramart; add the Maven wrapper (mvn wrapper:wrapper); first commit; create the GitHub repo; protect main.	AM-2
Wed 21 Oct	docker compose up -d; check every container is healthy; open Kafka UI; connect to catalog_db.	AM-5
Thu 22 Oct	Check the first CI run and make the build check required; first pull request: CONTRIBUTING.md, DEVLOG.md and the README badge.	AM-4
Fri 23 Oct	Create the GitHub Projects board and fields; create the Sprint 1 issues; add the Sprint 2 stories, later ones as titles.	AM-3
Sat 24 Oct	Start catalog-service once; refine Sprint 1 stories against the Definition of Ready; Sprint 0 review and retro.	—
Sun 25 Oct	Rest.	—

Sprint 0 · Setup & ways of working

Dates	19 Oct – 25 Oct 2026 · 1 week	**Commitment**	5 stories · 10 points
Sprint goal	**Everything needed to start building is ready: tools, repo, board, CI and running infrastructure.**		
Release	No release — setup only		

ID	User story	Acceptance criteria	Pts	Priority
AM-1	As a developer, I want all tools installed so that I can build and run AuraMart.	java -version shows 21; mvn -v, git --version, docker compose version and node -v all work.	2	Must
AM-2	As a developer, I want the Phase 1 scaffold in a GitHub repo with a protected main branch so that all work is versioned and reviewed.	Repo created and scaffold pushed; Maven wrapper added; main requires a pull request; PR template and CONTRIBUTING.md committed.	2	Must
AM-3	As the Scrum Master, I want a GitHub Projects board holding the backlog so that progress is always visible.	Status columns Backlog → Done; fields for Sprint, Points, Epic, Priority; Sprint 1–2 stories written in full, later ones as titles.	3	Must
AM-4	As a developer, I want CI to run on every push and pull request so that broken builds are caught immediately.	GitHub Actions runs mvn verify; a failing test fails the build; status badge shown in the README.	2	Must
AM-5	As a developer, I want the local infrastructure running so that services have Postgres, Redis and Kafka.	docker compose ps shows every container healthy; Kafka UI opens on :8090; can log in to catalog_db as catalog_svc.	1	Must

Key tasks & technical notes

- CI is moved forward from Phase 6 to Sprint 0 on purpose: catching a broken build on day one is far cheaper than on day 100. Sprint 12 extends it with Docker image builds.
- Write Sprint 1 and 2 stories in full now; keep later sprints as one-line titles and refine them just in time (that is normal Scrum, not laziness).

Sprint review demo: Clean clone → docker compose up → catalog-service starts → CI badge is green → board shows Sprint 1 ready.

Docs to write or update: README CI badge, CONTRIBUTING.md (branch, commit and PR rules), DEVLOG.md started.

> **Next**
>
> Sprint 1 starts on **Monday 26 October** with sprint planning — see Section 7.

7. Sprint planning (with Sprint 1 filled in)

What to bring to the meeting

- The product backlog, in order, with the top stories meeting the Definition of Ready.
- Your capacity: the hours you really have this sprint.
- Your velocity from the last three sprints (from Sprint 3 onwards).
- The improvement action from the last retrospective.

The 60-minute agenda

Step	Time	What you do
1. Look back	5 min	Last sprint's velocity and whether the retro action happened
2. Capacity	5 min	Hours available; subtract exams, travel, festivals, work crunches
3. Sprint goal	5 min	One sentence: the value this sprint delivers
4. Select	15 min	Pull Ready stories from the top of the backlog until capacity is reached
5. Plan tasks	25 min	Split each story into tasks of 4 hours or less and estimate the hours
6. Commit	5 min	Move the stories into the sprint; write the Day 1 DEVLOG entry

Sprint 1 planning — Monday 26 October, filled in

Capacity	Value
Available: 2 weeks × 15 h	30.0 h
Scrum events (planning, standups, refinement, review, retro)	− 3.0 h
Capacity for building	27.0 h
Starting guide: 27 h ÷ 1.5 h per point	≈ 18 points
Committed (a little slack for a first sprint)	17 points · 7 stories

> **Sprint goal**
>
> **The Catalog API runs end to end on local infrastructure, is tested in CI and documented.**

Task breakdown

Story	Task	Hours
AM-6 2 pts · 2.5 h	Read Flyway basics; walk through V1 and V2	1
	Fresh-volume test: compose down -v, up, start app; check flyway_schema_history	1
	Confirm Hibernate schema validation passes; log any fix in DEVLOG	0.5
AM-7 5 pts · 7.5 h	Walk through controller → service → repository code	1.5
	Run create / update / deactivate requests from catalog.http	1
	Fix any Spring Boot 4 compile or runtime issues found	2
	Verify 409 duplicate and 400 validation problem+json bodies	1

Story	Task	Hours
	Test that the SKU cannot change on update	1.5
	Pull request, self-review, squash merge	0.5
AM-8 3 pts · 3 h	Test category filter, sort and paging parameters	1
	Cap page size at 100 and test it	1
	Confirm inactive products are hidden; PR and merge	1
AM-9 2 pts · 2 h	Verify response fields; test inStock = false with the power bank	1.5
	Verify 404 problem+json for an unknown ID	0.5
AM-10 3 pts · 5 h	Extend service unit tests (update, deactivate)	1.5
	Controller tests for 201 / 400 / 404 / 409	3
	Confirm the tests run in CI	0.5
AM-11 1 pts · 1 h	Add OpenAPI title/version; check every endpoint is listed	0.5
	Check /actuator/health includes the database	0.5
AM-12 1 pts · 1.5 h	Fresh clone: follow the README exactly and fix gaps	1
	Add docs/adr/README.md index	0.5
Total	**7 stories**	**22.5 h**

22.5 h planned of 27 h available. The 4.5-hour gap is deliberate: a first sprint always brings surprises — new framework behaviour, setup problems, things to learn. If you don't need the buffer, use it to read ahead for Sprint 2 (Spring Cloud Gateway and Testcontainers docs); don't start Sprint 2 stories early.

Backlog refinement — Saturday of week 1

- Take the next sprint's stories and check each one against the Definition of Ready.
- Rewrite vague acceptance criteria until they can be tested.
- Split anything larger than 8 points.
- Re-estimate stories whose size changed because of what you learned this sprint.
- Note what you will need to learn, with links.

8. Sprint-by-sprint plan

One page per sprint. Each page is the input to that sprint's planning meeting: the goal, the stories with acceptance criteria and estimates, technical notes, what to show at the review and which documents to write. Treat later sprints as a forecast, not a promise — refine them as you learn, and re-plan from your real velocity after Sprint 2.

Sprint	Dates	Goal	Points	Release
S0	19 Oct – 25 Oct 2026	Setup & ways of working	10	—
S1	26 Oct – 8 Nov 2026	Catalog service	17	v0.1.0 ★
S2	9 Nov – 22 Nov 2026	API Gateway & real-infrastructure tests	17	v0.2.0
S3	23 Nov – 6 Dec 2026	Speed & search	18	v0.3.0 ★
S4	7 Dec – 20 Dec 2026	Orders & reliable events	19	v0.4.0
S5	21 Dec 2026 – 3 Jan 2027	Checkout saga	19	v0.5.0
S6	4 Jan – 17 Jan 2027	Notifications, idempotency & DLT	19	v0.6.0 ★
S7	18 Jan – 31 Jan 2027	Semantic search	18	v0.7.0
S8	1 Feb – 14 Feb 2027	AI shopping assistant	19	v0.8.0 ★
S9	15 Feb – 28 Feb 2027	Storefront I: browse, search, cart	18	v0.9.0
S10	1 Mar – 14 Mar 2027	Storefront II: checkout & AI chat	20	v0.10.0 ★
S11	15 Mar – 28 Mar 2027	Run everything & observe it	19	v0.11.0
S12	29 Mar – 11 Apr 2027	Secure, ship & scale	19	v0.12.0 ★
S13	12 Apr – 18 Apr 2027	Release & portfolio launch	8	v1.0.0 ★

★ = *milestone release.*

Sprint 1 · Catalog service

Dates	26 Oct – 8 Nov 2026 · 2 weeks	**Commitment**	7 stories · 17 points
Sprint goal	**The Catalog API runs end to end on local infrastructure, is tested in CI and documented.**		
Release	v0.1.0 ★ Milestone M1: Catalog API		

ID	User story	Acceptance criteria	Pts	Priority
AM-6	As a developer, I want the schema managed by Flyway with seed data so that every environment is identical.	On a fresh volume, app start applies V1 and V2; 12 seed products exist; Hibernate schema validation passes.	2	Must
AM-7	As an admin, I want to create, update and deactivate products so that I can manage the catalogue.	POST → 201 + Location; PUT updates; DELETE → 204 and product hidden; duplicate SKU → 409; invalid body → 400 problem+json.	5	Must
AM-8	As a shopper, I want to browse products by category with paging and sorting so that I find items quickly.	?category=kitchen&sort=price,asc&size=5 returns correct page metadata; inactive products hidden; page size capped at 100.	3	Must
AM-9	As a shopper, I want to see price and stock status so that I know whether I can buy a product.	GET /products/{id} returns price, currency, stockQuantity and inStock; unknown ID → 404 problem+json.	2	Must
AM-10	As a developer, I want automated tests for the catalog so that changes don't silently break it.	Service unit tests (create, duplicate, not found, update); controller tests for 201/400/404/409; all green in CI.	3	Must
AM-11	As an API consumer, I want Swagger UI and health endpoints so that the API is discoverable and monitorable.	/swagger-ui.html lists every endpoint with API title and version; /actuator/health shows UP including the database.	1	Should
AM-12	As a reviewer, I want ADRs and verified run instructions so that decisions and setup are documented.	ADR 0001–0003 in docs/adr with an index; README steps work exactly as written on a fresh clone.	1	Must

Key tasks & technical notes

- Run the scaffold first and fix anything Spring Boot 4 changed (starter names, test annotations). Write every fix in DEVLOG.md — that list is great interview material.
- Cap page size with spring.data.web.pageable.max-page-size=100 so no client can request a million rows.
- Use services/catalog-service/catalog.http to demonstrate every acceptance criterion.
- The seeded power bank has zero stock — use it to test inStock = false.

Sprint review demo: Create → duplicate (409) → invalid (400) → list by category → deactivate → GET returns 404. Show the green CI run.

Docs to write or update: docs/01-requirements/user-stories.md (Sprint 1 stories), ADR index, README run steps verified.

Sprint 2 · API Gateway & real-infrastructure tests

Dates	9 Nov – 22 Nov 2026 · 2 weeks	**Commitment**	5 stories · 17 points
Sprint goal	**All traffic enters through one gateway, and tests run against real PostgreSQL instead of fakes.**		
Release	v0.2.0		

ID	User story	Acceptance criteria	Pts	Priority
AM-13	As a client app, I want a single entry point on :8080 so that the frontend only needs one URL.	localhost:8080/api/v1/products returns the same as :8081; routes defined in application.yml; unknown route → 404.	5	Must
AM-14	As a frontend developer, I want CORS configured at the gateway so that the browser app on :5173 can call the API.	Preflight from http://localhost:5173 is allowed; other origins are rejected; verified with curl.	2	Must
AM-15	As a developer, I want a request ID on every call so that I can follow one request through the logs.	Gateway adds X-Request-Id when missing; it is returned in the response; catalog log lines print it (MDC).	3	Should
AM-16	As a developer, I want integration tests against real PostgreSQL so that tests behave like production.	Testcontainers starts the pgvector image; Flyway runs; repository and API tests pass locally and in CI; no H2.	5	Must
AM-17	As a developer, I want test coverage reports so that I can see untested code.	JaCoCo report produced in CI; catalog-service line coverage ≥ 70%.	2	Should

Key tasks & technical notes

- Use the Spring Cloud Gateway Server WebMVC starter so the gateway matches the servlet stack of the other services; add it as a new module in the parent POM.
- @ServiceConnection wires Testcontainers into Spring Boot without hand-written URLs or ports.
- Request IDs are a stepping stone: in Sprint 11 OpenTelemetry trace IDs take over, and you will see why they are better.
- Record ADR-0004: why a WebMVC gateway and not the reactive (WebFlux) one.

Sprint review demo: One curl through :8080 showing the X-Request-Id header, the same ID in the catalog log, and the CI coverage report.

Docs to write or update: ADR-0004, high-level design updated with the gateway.

Sprint 3 · Speed & search

Dates	23 Nov – 6 Dec 2026 · 2 weeks	**Commitment**	5 stories · 18 points
Sprint goal	**Product reads are fast thanks to Redis, and search still works when shoppers make typos.**		
Release	v0.3.0 ★ Milestone M2: Gateway, caching & search		

ID	User story	Acceptance criteria	Pts	Priority
AM-18	As a shopper, I want product pages to load instantly so that browsing feels fast.	GET /products/{id} cached in Redis (cache-aside, TTL 10 min); the second call is a cache hit, visible in logs and redis-cli.	5	Must
AM-19	As an admin, I want my edits to show immediately so that shoppers never see stale prices.	PUT and DELETE evict the cached product; a Testcontainers test (Postgres + Redis) proves there is no stale read.	3	Must
AM-20	As a shopper, I want typo-tolerant keyword search so that 'hedphones' still finds headphones.	GET /api/v1/products/search?q= ranks by pg_trgm similarity; best match first; blank q → 400.	5	Must
AM-21	As a shopper, I want the store to keep working if the cache fails so that a Redis outage isn't a store outage.	With the Redis container stopped, the product API still answers from the database and logs a warning.	3	Should
AM-22	As the team, I want a test plan so that the testing strategy is written down and agreed.	docs/04-testing/test-plan.md covers unit, integration, end-to-end and performance tests, with tools and pass criteria.	2	Should

Key tasks & technical notes

- @Cacheable / @CacheEvict on ProductService, with spring.cache.redis.time-to-live: 10m.
- Cache response records (ProductResponse), never JPA entities.
- Fail-safe caching = a custom CacheErrorHandler that logs the error and lets the call fall through to the database.
- Search: ORDER BY similarity(name, :q) DESC with a minimum threshold; the GIN trigram index from V1 keeps it fast.
- Record ADR-0005: cache-aside with TTL plus explicit eviction.

Sprint review demo: First vs. second response time → edit a price and see it immediately → stop Redis, still works → search 'hedphones'.

Docs to write or update: docs/04-testing/test-plan.md, ADR-0005. Tag v0.3.0 = milestone M2.

Sprint 4 · Orders & reliable events

Dates	7 Dec – 20 Dec 2026 · 2 weeks	**Commitment**	4 stories · 19 points
Sprint goal	**Shoppers can place an order, and the OrderPlaced event is guaranteed to reach Kafka.**		
Release	v0.4.0		

ID	User story	Acceptance criteria	Pts	Priority
AM-23	As a developer, I want shared event contracts so that producers and consumers agree on event formats.	libs/event-contracts holds records for all 5 events (eventId, occurredAt, version); JSON round-trip test; docs/03-design/events.md written.	3	Must
AM-24	As a shopper, I want a cart so that I can collect items before ordering.	Create cart, add item, change quantity, remove item; name and price copied from Catalog when added; stored in order_db.	3	Must
AM-25	As a shopper, I want to place an order from my cart so that I can buy products.	POST /api/v1/orders → 201 with status PENDING; order lines saved; gateway routes /api/v1/orders/** and /api/v1/carts/**.	5	Must
AM-26	As the business, I want OrderPlaced never to be lost so that no order goes unprocessed.	Order and outbox row saved in one transaction; relay publishes to order.events with key = orderId; stop Kafka, place order, restart → event still arrives.	8	Must

Key tasks & technical notes

- Create order-service (port 8082, order_db, Flyway). Until login exists (Sprint 12), identify the shopper with an X-Customer-Id header.
- Order reads price and stock from Catalog with RestClient — a synchronous read is fine here; the writes go through events.
- Outbox relay: a @Scheduled poller reads unsent rows (FOR UPDATE SKIP LOCKED), sends them with KafkaTemplate, then marks them sent.
- Declare topics in code with NewTopic beans (3 partitions) — the broker refuses auto-created topics.
- Record ADR-0006: transactional outbox vs. publishing to Kafka straight after the commit.

Sprint review demo: Place an order → row appears in the outbox table → message in Kafka UI with key = orderId → repeat with Kafka stopped.

Docs to write or update: docs/03-design/events.md (event catalogue), order_db ERD, ADR-0006.

Sprint 5 · Checkout saga

Dates	21 Dec 2026 – 3 Jan 2027 · 2 weeks	**Commitment**	4 stories · 19 points
Sprint goal	**Every order automatically ends CONFIRMED or CANCELLED, and stock always stays correct.**		
Release	v0.5.0		

ID	User story	Acceptance criteria	Pts	Priority
AM-27	As a shopper, I want stock reserved when I order so that my items aren't sold to someone else.	Catalog consumes OrderPlaced, reserves stock with optimistic locking, publishes StockReserved; two orders for the last unit → only one reserved (test).	8	Must
AM-28	As a shopper, I want my order cancelled if stock runs out so that I'm not charged for unavailable items.	Insufficient stock → StockRejected; Order sets CANCELLED with the reason.	3	Must
AM-29	As a shopper, I want payment taken after stock is reserved so that my order gets confirmed.	Order consumes StockReserved → simulated payment → CONFIRMED + OrderConfirmed, or CANCELLED + OrderCancelled on failure.	5	Must
AM-30	As the business, I want reserved stock released when payment fails so that inventory stays correct.	Catalog consumes OrderCancelled and returns the stock; an integration test shows stock back at its original value.	3	Must

Key tasks & technical notes

- Simulated PaymentGateway: a configurable failure rate plus an 'always fail' switch for demos and tests.
- Saga consumers use blocking retries (DefaultErrorHandler) so events for one order are never processed out of order.
- Draw the saga sequence diagram and record ADR-0007: choreography vs. orchestration.
- Holiday sprint (21 Dec – 3 Jan): if your capacity is lower, commit fewer points in planning. The sprint dates never move.

Sprint review demo: One successful checkout and one failed payment, watching status and stock change in Kafka UI and the database.

Docs to write or update: Saga sequence diagram in docs/02-architecture, ADR-0007.

Sprint 6 · Notifications, idempotency & DLT

Dates	4 Jan – 17 Jan 2027 · 2 weeks	**Commitment**	5 stories · 19 points
Sprint goal	**Failures are handled safely: retries, a Dead Letter Topic, and no duplicate side effects.**		
Release	v0.6.0 ★ Milestone M3: Event-driven checkout complete		

ID	User story	Acceptance criteria	Pts	Priority
AM-31	As a shopper, I want a confirmation or cancellation message so that I know my order's outcome.	notification-service (port 8084) consumes OrderConfirmed and OrderCancelled and sends a message (log, or local email via Mailpit).	3	Must
AM-32	As an operator, I want failed notifications retried and then parked in a DLT so that one bad message never blocks the queue.	Simulated failure → retries with backoff → message lands in order.events-dlt with error headers; later messages keep flowing.	5	Must
AM-33	As the business, I want duplicate events ignored so that stock isn't reserved twice and emails aren't sent twice.	processed_events table in each consumer; replaying the same event causes no second side effect (test).	5	Must
AM-34	As a shopper, I want to see my orders and their status so that I can track them.	GET /api/v1/orders/{id} and GET /api/v1/orders show status and timestamps (PENDING → CONFIRMED or CANCELLED).	3	Must
AM-35	As a developer, I want an end-to-end saga test so that regressions in the event flow are caught.	Testcontainers (Kafka + Postgres) test covers the happy path and a payment failure; runs in CI.	3	Should

Key tasks & technical notes

- Notifications use non-blocking retries (@RetryableTopic). Retried messages can arrive out of order — fine for emails, never acceptable for the saga.
- Mailpit is an optional extra container that catches emails locally, so nothing is really sent.
- Store the processed event ID in the same database transaction as the side effect — otherwise idempotency has a gap.
- Write the runbook section: how to inspect and replay a DLT message.

Sprint review demo: Flow D live: break the notifier, watch the retries, find the message in the DLT, fix it, replay it.

Docs to write or update: Runbook: DLT handling. Tag v0.6.0 = milestone M3 (the biggest one).

Sprint 7 · Semantic search

Dates	18 Jan – 31 Jan 2027 · 2 weeks	**Commitment**	4 stories · 18 points
Sprint goal	**Products can be found by meaning, using embeddings stored in pgvector.**		
Release	v0.7.0		

ID	User story	Acceptance criteria	Pts	Priority
AM-36	As a developer, I want a working LLM and embedding setup so that AI features run locally or on a hosted API.	Ollama profile with an embedding model and a tool-calling chat model, or a hosted API key; switch by Spring profile; spending limit set if hosted.	2	Must
AM-37	As a shopper, I want to search by meaning so that 'something to keep tea hot on a train' finds the steel flask.	Catalog stores product embeddings in pgvector; GET /api/v1/products/semantic-search?q= returns the flask in the top 3.	8	Must
AM-38	As an admin, I want embeddings updated when I change a product so that search is always current.	Create/update re-embeds the product asynchronously; deactivated products are removed from the vector store.	5	Must
AM-39	As a shopper, I want exact names and meanings both to work so that 'Aura Pods' and 'gym earbuds' each find the right item.	Hybrid ranking combines trigram and vector scores; an exact name match ranks first. Stretch story.	3	Could

Key tasks & technical notes

- Embed name + category + description as one text per product; keep the product ID in the metadata.
- The vector dimension must match the embedding model (768 for nomic-embed-text, 1536 for text-embedding-3-small). Changing model = re-embedding everything.
- Record ADR-0008: which embedding model and why.
- The hybrid-search story is marked Could: drop it first if the sprint runs late.

Sprint review demo: Keyword vs. semantic results side by side for the same natural-language question.

Docs to write or update: ADR-0008.

Sprint 8 · AI shopping assistant

Dates	1 Feb – 14 Feb 2027 · 2 weeks	**Commitment**	6 stories · 19 points
Sprint goal	**A chat assistant recommends real products and answers order questions.**		
Release	v0.8.0 ★ Milestone M4: AI shopping assistant		

ID	User story	Acceptance criteria	Pts	Priority
AM-40	As a shopper, I want to ask for products in plain language so that I can shop by chatting.	ai-assistant-service (port 8083) exposes POST /api/v1/assistant/chat using Spring AI ChatClient with a shopping-assistant system prompt.	3	Must
AM-41	As a shopper, I want answers based only on the real catalogue so that the assistant never invents products.	searchProducts tool calls Catalog semantic search; replies mention only returned products; unknown items → 'not available'.	5	Must
AM-42	As a shopper, I want to ask 'where is my order?' so that I get the real status.	getOrderStatus tool calls the Order service; unknown order IDs are handled politely.	3	Must
AM-43	As a shopper, I want the assistant to remember our conversation so that 'show me a cheaper one' works.	Conversation ID per chat; memory stored in assistant_db; survives a service restart.	3	Must
AM-44	As a shopper, I want replies to stream word by word so that the chat feels fast.	POST /api/v1/assistant/chat/stream returns text/event-stream; tokens arrive progressively (curl -N).	3	Must
AM-45	As the owner, I want guardrails so that the assistant stays a shopping assistant.	Refuses unrelated tasks; ignores instructions hidden inside product text; every tool is read-only.	2	Must

Key tasks & technical notes

- Tools are plain Java methods annotated with @Tool that call Catalog and Order through RestClient.
- Keep a 'golden questions' list (10 prompts + expected behaviour) and re-run it after every prompt change.
- Record the model, its limits and the system prompt in docs/03-design/prompt-design.md.

Sprint review demo: The laptop-bag conversation, a 'cheaper one' follow-up, an order-status question and a polite off-topic refusal.

Docs to write or update: docs/03-design/prompt-design.md (system prompt, tools, golden questions). Tag v0.8.0 = milestone M4.

Sprint 9 · Storefront I: browse, search, cart

Dates	15 Feb – 28 Feb 2027 · 2 weeks	**Commitment**	5 stories · 18 points
Sprint goal	**Shoppers can browse, search and fill a cart in the React app.**		
Release	v0.9.0		

ID	User story	Acceptance criteria	Pts	Priority
AM-46	As a developer, I want the React project set up so that frontend work has a solid base.	frontend/auramart-web with Vite, TypeScript, Tailwind, React Router and TanStack Query; gateway URL from .env; lint and build in CI.	3	Must
AM-47	As a shopper, I want a product list with category filter, sorting and paging so that I can browse.	Data comes through the gateway; filter, sort and page are reflected in the URL; loading and empty states shown.	5	Must
AM-48	As a shopper, I want a search bar so that I can find products by keyword or by meaning.	Debounced input; keyword/semantic toggle; results page.	3	Must
AM-49	As a shopper, I want a product page so that I can see details, price and stock.	/products/:id shows details; out-of-stock disables Add to cart.	2	Must
AM-50	As a shopper, I want a cart page so that I can review and change my items.	Add, update and remove synced with the cart API; totals shown; cart survives a page refresh.	5	Must

Key tasks & technical notes

- Simple feature folders: pages/, components/, api/, hooks/.
- Generate TypeScript types from the OpenAPI spec (for example with openapi-typescript) so frontend and backend can't drift apart.
- Build mobile-first with Tailwind from the first component.

Sprint review demo: Browse → search → product page → cart, at desktop and phone width.

Docs to write or update: frontend/README.md, first UI screenshots.

Sprint 10 · Storefront II: checkout & AI chat

Dates	1 Mar – 14 Mar 2027 · 2 weeks	**Commitment**	5 stories · 20 points
Sprint goal	**Checkout, order tracking and the AI chat panel all work in the browser.**		
Release	v0.10.0 ★ Milestone M5: MVP: full storefront		

ID	User story	Acceptance criteria	Pts	Priority
AM-51	As a shopper, I want to check out and watch my order status so that I know it went through.	Place order from the cart; status polls every 2 s until CONFIRMED or CANCELLED; a clear message for each outcome.	5	Must
AM-52	As a shopper, I want an order history page so that I can see past orders.	List with status badges; clicking opens the order detail.	2	Must
AM-53	As a shopper, I want an AI chat panel so that I can ask for products while browsing.	Side drawer; streaming replies; product names link to product pages; conversation kept while navigating.	8	Must
AM-54	As a shopper, I want clear loading, empty and error states so that the app never looks broken.	Every page has the three states; layout works at 360 px width.	3	Should
AM-55	As a developer, I want frontend tests so that UI changes don't break key flows.	Vitest + React Testing Library tests for cart and chat components; run in CI.	2	Should

Key tasks & technical notes

- The browser's EventSource only supports GET. For the POST streaming endpoint, read the response with fetch() and a ReadableStream reader.
- Ask the assistant to return product IDs (or a small JSON block) so the UI can render clickable product cards.
- MVP milestone: after this sprint all four end-to-end flows (A–D) work through the UI.

Sprint review demo: The full interview story through the UI (everything except the trace).

Docs to write or update: README feature tour with a GIF. Tag v0.10.0 = milestone M5 (MVP).

Sprint 11 · Run everything & observe it

Dates	15 Mar – 28 Mar 2027 · 2 weeks	**Commitment**	4 stories · 19 points
Sprint goal	**The whole system starts with one command, and every request can be traced.**		
Release	v0.11.0		

ID	User story	Acceptance criteria	Pts	Priority
AM-56	As a developer, I want every service containerised so that the whole system starts with one command.	Dockerfile per service (layered jars or buildpacks); docker compose --profile app up starts everything; frontend served by nginx.	5	Must
AM-57	As an operator, I want distributed tracing so that I can follow one checkout across services and Kafka.	OpenTelemetry export to a local Grafana stack; one checkout shows a single trace: gateway → order → Kafka → catalog → notification.	8	Must
AM-58	As an operator, I want metrics dashboards so that I can see system health at a glance.	Grafana dashboard with request rate, latency, error rate and Kafka consumer lag.	3	Must
AM-59	As a shopper, I want the app to degrade gracefully when a service is slow so that pages don't hang.	Resilience4j timeout + circuit breaker on Order → Catalog and on assistant tool calls, with friendly fallbacks.	3	Should

Key tasks & technical notes

- grafana/otel-lgtm is a single container with an OpenTelemetry collector, Prometheus, Tempo, Loki and Grafana — light enough for a laptop.
- Turn on Kafka observation (spring.kafka.template.observation-enabled and spring.kafka.listener.observation-enabled) so traces continue across Kafka.
- Give every container a memory limit and size the JVM to it (for example -XX:MaxRAMPercentage=75).

Sprint review demo: The interview story with the matching trace open in Grafana.

Docs to write or update: docs/05-operations/observability.md: how to read a trace.

Sprint 12 · Secure, ship & scale

Dates	29 Mar – 11 Apr 2027 · 2 weeks	**Commitment**	3 stories · 19 points
Sprint goal	**AuraMart is secured, built by CI and deployed on Kubernetes.**		
Release	v0.12.0 ★ Milestone M6: Production-ready		

ID	User story	Acceptance criteria	Pts	Priority
AM-60	As a shopper, I want to log in so that my cart and orders are private.	Keycloak realm with shopper and admin roles; React login (OIDC + PKCE); gateway validates the JWT; customer ID comes from the token; product admin needs the admin role.	8	Must
AM-61	As a developer, I want CI to build and publish images so that every commit on main is deployable.	GitHub Actions builds, tests, builds Docker images and pushes them to GitHub Container Registry on main.	3	Must
AM-62	As an operator, I want AuraMart on Kubernetes so that it runs and scales like production.	kind cluster; Deployments, Services, ConfigMaps, Secrets; liveness/readiness probes on actuator; HPA scales catalog-service under load.	8	Must

Key tasks & technical notes

- Keycloak runs as one more container; export the realm to JSON and commit it so the setup is repeatable.
- The HPA needs metrics-server installed in the kind cluster. Generate load with a tool such as k6 or hey.
- Kubernetes is RAM-hungry: stop the Compose app containers and use a hosted LLM while testing on kind.
- Record ADR-0009: Keycloak + token validation at the gateway.

Sprint review demo: Log in, place an order, then watch kubectl get hpa while a load test scales catalog-service.

Docs to write or update: docs/05-operations/deployment.md, security notes, ADR-0009. Tag v0.12.0 = milestone M6.

Sprint 13 · Release & portfolio launch

Dates	12 Apr – 18 Apr 2027 · 1 week	**Commitment**	4 stories · 8 points
Sprint goal	**AuraMart v1.0.0 is released, documented and ready to show.**		
Release	v1.0.0 ★ Milestone M7: Release & portfolio launch		

ID	User story	Acceptance criteria	Pts	Priority
AM-63	As the owner, I want a bug bash so that v1.0 ships with no known critical bugs.	Run flows A–D and the golden AI questions end to end; fix every critical bug; log the rest as issues.	3	Must
AM-64	As a reviewer, I want complete documentation so that anyone can run and understand AuraMart.	Runbook, CHANGELOG and v1.0.0 release notes; architecture docs and ADR index current; README quick start verified.	2	Must
AM-65	As a recruiter, I want a short demo video so that I can see the system working in 5 minutes.	5-minute video of the interview story, including the trace; linked from the README.	2	Must
AM-66	As a job seeker, I want the project presented well so that it helps my career.	README screenshots/GIF; GitHub release v1.0.0; resume bullets and a LinkedIn post published.	1	Must

Key tasks & technical notes

- Feature freeze: no new features this sprint — only fixes, docs and presentation.
- Use the v1.0 checklist and resume bullets in Section 12.

Sprint review demo: The 5-minute video itself is the sprint review.

Docs to write or update: Runbook complete, CHANGELOG.md, v1.0.0 release notes. Tag v1.0.0 = milestone M7.

9. Tracking progress

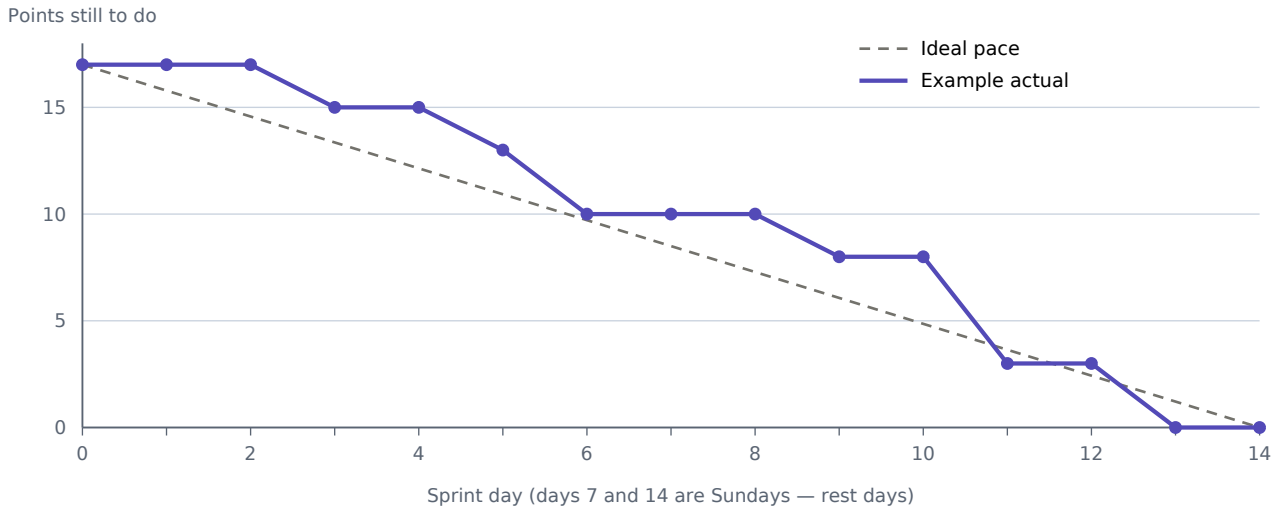
Daily standup — DEVLOG.md

```
## Tue 27 Oct 2026 · Sprint 1 · Day 2
Done yesterday:  AM-6 Flyway verified on a fresh volume
Today:           AM-7 run every catalog.http request; check the 409 body
Blockers:        none
Learned:         Flyway records each migration in flyway_schema_history
```

Two minutes, at the start of every session. The 'Learned' lines become your interview stories later.

Burndown chart

At the end of each day, note the points still to do. The dashed line is the ideal pace. Your real line will be bumpy because points only drop when a story is fully Done — that is normal. A line that stays flat for days is the signal to act.



Illustrative example for Sprint 1 (17 points). Draw yours from real numbers; GitHub Projects' Insights can chart the same data.

Velocity tracker

Sprint	Planned	Completed	Carried over	Notes
S0	10			
S1	17			
S2	17			
S3	18			
S4	19			
S5	19			
S6	19			
S7	18			
S8	19			
S9	18			
S10	20			
S11	19			
S12	19			
S13	8			

From Sprint 3 onwards, plan with the average completed points of the last three sprints.

Sprint review checklist

- ☐ Demo every Done story against its acceptance criteria.
- ☐ Record a 2–3 minute screen video of the demo.
- ☐ Update the README (features, screenshots).
- ☐ Tag the release (git tag vX.Y.0) and write short release notes.
- ☐ Put unfinished stories back in the backlog and re-estimate them — no partial credit.
- ☐ Record the completed points in the velocity tracker.

Retrospective template

```
## Sprint 1 retrospective · Sat 7 Nov 2026
Energy (1-5):      4
What went well:    ...
What didn't:       ...
What I learned:    ...
Last sprint's action done?  yes / no
One action for next sprint (small, specific):
  -> e.g. "Write the integration test before the code in AM-16"
```

When the sprint goes wrong

- **A story is bigger than you thought** → split it, finish the most valuable slice, return the rest to the backlog.
- **Blocked for more than a day** → write the blocker in the DEVLOG, move to the next story, come back with fresh eyes.
- **Finished early** → improve tests or docs, or read ahead; don't start the next sprint's stories.
- **Missed the sprint goal** → no blame. Find the cause in the retro and lower the next commitment.

10. Risks and how to handle them

Risk	Likelihood	Impact	What to do
Learning takes longer than estimated	High	Medium	Measure real velocity after Sprint 2 and re-plan; drop Could stories first (AM-39); split anything over 8 points.
The laptop runs out of RAM	Medium	Medium	Use a hosted LLM; run only the services a sprint needs; set JVM and container memory limits.
Tutorials don't match Spring Boot 4 / Spring AI 2	High	Low	Use the official reference docs first; check the version of every tutorial; log fixes in the DEVLOG.
Scope creep (“just one more feature”)	Medium	High	New ideas go to the Later column; only sprint planning adds work to a sprint.
Life gets busy (work, exams, festivals, illness)	High	Medium	Lower the commitment in planning, never the sprint length; keep the retro even after a bad sprint.
Distributed bugs are hard to find	Medium	Medium	Request IDs (Sprint 2), Kafka UI, end-to-end tests (Sprint 6), tracing (Sprint 11).
An unexpected API bill	Low	Medium	Ollama by default; a spending limit and a budget alert before any hosted API key.
Burnout or losing motivation	Medium	High	Sunday off; demo something every sprint; share each milestone publicly.

11. Documentation plan

Documentation is written in the sprint where the work happens, not at the end. This is the end-to-end set a real team produces, scaled down for one person.

Sprint	Documents
S0	README CI badge, CONTRIBUTING.md (branch, commit and PR rules), DEVLOG.md started.
S1	docs/01-requirements/user-stories.md (Sprint 1 stories), ADR index, README run steps verified.
S2	ADR-0004, high-level design updated with the gateway.
S3	docs/04-testing/test-plan.md, ADR-0005. Tag v0.3.0 = milestone M2.
S4	docs/03-design/events.md (event catalogue), order_db ERD, ADR-0006.
S5	Saga sequence diagram in docs/02-architecture, ADR-0007.
S6	Runbook: DLT handling. Tag v0.6.0 = milestone M3 (the biggest one).
S7	ADR-0008.
S8	docs/03-design/prompt-design.md (system prompt, tools, golden questions). Tag v0.8.0 = milestone M4.
S9	frontend/README.md, first UI screenshots.
S10	README feature tour with a GIF. Tag v0.10.0 = milestone M5 (MVP).
S11	docs/05-operations/observability.md: how to read a trace.
S12	docs/05-operations/deployment.md, security notes, ADR-0009. Tag v0.12.0 = milestone M6.
S13	Runbook complete, CHANGELOG.md, v1.0.0 release notes. Tag v1.0.0 = milestone M7.

```
docs/
├── 01-requirements/   user-stories.md, non-functional.md
├── 02-architecture/   hld.md, diagrams/
├── 03-design/         events.md, erd.md, prompt-design.md
├── 04-testing/        test-plan.md
├── 05-operations/     runbook.md, observability.md, deployment.md
├── adr/               README.md (index), 0001-… to 0009-…
└── DEVLOG.md          daily standups and retrospectives
CHANGELOG.md
```

12. Launch and beyond

v1.0 release checklist

- ☐ All four end-to-end flows work through the UI.
- ☐ CI is green on main and Docker images are published.
- ☐ Both the one-command Compose start and the kind deployment are documented.
- ☐ README: what and why, architecture image, quick start, demo GIF, tech stack.
- ☐ The 5-minute demo video is linked from the README.
- ☐ GitHub release v1.0.0 has release notes.
- ☐ Resume and LinkedIn are updated.

Resume bullet templates

Replace with your own wording and real numbers once you have measured them.

- Designed and built AuraMart, an event-driven e-commerce platform: 5 Spring Boot 4 services, Apache Kafka, PostgreSQL with pgvector, Redis and React.
- Implemented a choreography saga with a transactional outbox, idempotent consumers, retries and a Dead Letter Topic, keeping orders and stock consistent when payments fail.
- Built an AI shopping assistant with Spring AI: RAG over product embeddings in pgvector, tool calling to live order data, chat memory and streaming replies.
- Added OpenTelemetry tracing, Grafana dashboards, Keycloak security, GitHub Actions CI and a Kubernetes (kind) deployment with autoscaling.

The 2-minute interview demo

1. Ask the assistant for a laptop bag; show it recommending a real product.
2. Open the product and add it to the cart.
3. Switch the payment simulator to 'always fail' and check out.
4. Show the order CANCELLED and the stock returned (compensation).
5. Show the notification retries and the message waiting in the DLT in Kafka UI.
6. Open the trace in Grafana: one request across every service and Kafka.

After v1.0 — the 'Later' backlog

Idea	What you would learn
Kafka Streams “top sellers” leaderboard	Stream processing — the Kafka Streams item on your roadmap
Debezium change data capture for the outbox	Replacing the polling relay with CDC
Public demo on one AWS EC2 instance (free credits)	Cloud deployment and cost control
MCP server exposing catalogue search	Letting other AI clients use your tools
Payment provider sandbox	Real payment APIs and webhooks
Admin UI for products and orders	Role-based screens and forms

Appendix A. Git workflow

Branch names

Type	Pattern	Example
Feature	feature/AM-<id>-<short-name>	feature/AM-7-product-crud
Bug fix	fix/AM-<id>-<short-name>	fix/AM-31-email-encoding
Docs	docs/AM-<id>-<short-name>	docs/AM-22-test-plan
Chore	chore/<short-name>	chore/upgrade-spring-boot

Commit messages (Conventional Commits)

Type	Use for	Example
feat	A new feature	feat(catalog): add product update endpoint (AM-7)
fix	A bug fix	fix(order): stop double-publishing outbox rows (AM-26)
test	Tests only	test(catalog): add duplicate SKU test (AM-10)
docs	Documentation	docs(adr): add ADR-0006 transactional outbox (AM-26)
refactor	Restructure, no behaviour change	refactor(catalog): extract ProductMapper
chore	Build, dependencies, tooling	chore: add Maven wrapper (AM-2)
ci	CI pipeline	ci: run mvn verify on pull requests (AM-4)

Daily flow and release tagging

```
git switch main && git pull
git switch -c feature/AM-7-product-crud
# ...work in small steps, commit often...
git add -A
git commit -m "feat(catalog): add product update endpoint (AM-7)"
git push -u origin feature/AM-7-product-crud
# open a PR on GitHub -> CI green -> self-review -> squash merge -> delete branch

# at every sprint review: tag the release
git switch main && git pull
git tag -a v0.1.0 -m "Sprint 1: Catalog API"
git push origin v0.1.0
# then on GitHub: Releases -> Draft a new release from the tag, paste the release notes
```

Pull request template (.github/pull_request_template.md)

```
## What
## Why (story)
Closes #<issue-number>
## How I tested it
## Screenshots / output
## Definition of Done
- [ ] Tests added and passing; CI green
- [ ] Every acceptance criterion demonstrated
- [ ] Docs / ADR / events.md updated if needed
- [ ] No secrets, no TODO without an issue
```